

Comparative Study of Object Tracking Algorithms: KCF, CSRT, and TLD

Your Name

April 11, 2026

Contents

1	Introduction	3
2	Literature Review	4
2.1	KCF	4
2.2	CSRT	5
2.3	TLD	6
2.4	KCF-TLD	6
3	Methodology	6
3.1	Proposed KCF-TLD Algorithm	6
3.2	KCF as the Base Tracker	7
3.3	TLD as the Recovery Tracker	7
3.4	Hybrid Recovery Strategy	7
3.4.1	Design Logic	8
3.4.2	Triggering TLD	8
3.4.3	Controlling TLD Overhead	9
3.4.4	Validating TLD Recovery	9
4	Results and Analysis	11
4.1	Performance Metrics	11
4.2	Algorithm Comparison	11

5	Discussion	11
6	Conclusion	13

1 Introduction

Under the rapid development of the low-altitude economy, the demand for UAV tracking modules continues to rise in both industry and research. Classical discriminative trackers such as KCF, CSRT, and TLD have already provided important technical foundations for practical target tracking. KCF, proposed by Henriques et al., relies on circulant matrices and the kernel trick, and uses the fast Fourier transform in the frequency domain to achieve very efficient computation^[10]. Later studies further improved correlation-filter tracking through multi-template scale adaptation, spatial regularisation, and spatial reliability modelling^[3,6,17]. CSRT, proposed by Lukežič et al., introduces channel reliability and spatial reliability so as to improve robustness while maintaining good accuracy^[14]. TLD, proposed by Kalal et al., combines tracking, learning, and detection, and adapts to target appearance variation through online updating^[11]. However, each of these algorithms has an obvious shortcoming. KCF is extremely fast and has low computational cost, but it can easily lose the target after short-term occlusion. CSRT has higher precision, stronger robustness in complicated scenes, and better scale adaptation, but its computational cost increases significantly as its precision improves. TLD has persistent learning capability and good resistance to occlusion, but its heavy computational burden makes real-time performance difficult to maintain^[11].

Although many studies have focused on UAV tracking, most existing work concentrates on improving a single algorithm rather than carrying out a systematic comparison of several classical algorithms under a unified benchmark. In particular, in UAV tracking, the target often undergoes drastic viewpoint change, the platform shakes frequently, and the available computational resources are limited. Recent UAV-oriented and real-time KCF studies also show that practical deployment depends heavily on the balance between robustness and cost rather than on one metric alone^[7,15]. Therefore, the trade-off among accuracy, speed, and computational cost still requires more explicit analysis. Based on this background, this paper systematically compares KCF, CSRT, and TLD on standard datasets, quantitatively analyses their application boundaries in UAV tracking, and attempts to provide a clearer basis for algorithm selection and later optimisation.

At the same time, this project further proposes a hybrid KCF–TLD method. The aim is not simply to force two algorithms to run together, but to combine their advantages in a role-based way: KCF is used as the main tracker because of its speed, while TLD is used

as a repair module when the target is lost. In this way, the study not only compares the three classical algorithms, but also explores whether a complementary design can achieve a more practical balance between high frame rate and target recovery capability.

2 Literature Review

According to the core mechanism of modern discriminative trackers, the algorithms discussed in this paper can be roughly divided into three categories: kernel correlation filter methods represented by KCF, channel and spatial reliability methods represented by CSRT, and long-term tracking-learning-detection methods represented by TLD^[1]. At the same time, the broader tracking literature has continued to extend these directions through spatio-temporal context learning, patch reliability, complementary online learners, and later deep or Siamese-based tracking frameworks^[2,5,8-9,12-13,19].

2.1 KCF

The core idea of KCF is to train a discriminative classifier through a kernelized correlation filter. It generates a large set of translated samples by cyclic shifts of the target patch and learns the model by ridge regression. Its core frequency-domain solution can be written as

$$\hat{w} = \frac{\hat{x}^* \odot \hat{y}}{\hat{x}^* \odot \hat{x} + \lambda},$$

where $\hat{x}$ denotes the Fourier transform of the sample, $\hat{y}$ is the desired Gaussian response, and λ is the regularisation parameter. By diagonalising circulant matrices through the discrete Fourier transform, KCF greatly simplifies computation and therefore achieves a very high frame rate^[10]. Its main strengths are speed and relatively low computational demand. However, its weaknesses are also clear: it is prone to bounding-box drift, its ability to handle large scale change is limited, and it usually loses the target after severe occlusion. Later variants have tried to improve KCF by using deep features, Siamese features, and richer scale modelling^[4,16-17,20].

Table 1: Matlab code, with a Gaussian kernel (Core KCF logic)

Inputs

x : training image patch, $m \times n \times c$
 y : regression target, Gaussian-shaped, $m \times n$
 z : test image patch, $m \times n \times c$

Output

responses: detection score for each location, $m \times n$

```
function alphaf = train(x, y, sigma, lambda)
    k = kernel_correlation(x, x, sigma);
    alphaf = fft2(y) ./ (fft2(k) + lambda);
end

function responses = detect(alphaf, x, z, sigma)
    k = kernel_correlation(z, x, sigma);
    responses = real(ifft2(alphaf .* fft2(k)));
end

function k = kernel_correlation(x1, x2, sigma)
    c = ifft2(sum(conj(fft2(x1)) .* fft2(x2), 3));
    d = x1(:)' * x1(:) + x2(:)' * x2(:) - 2 * c;
    k = exp(-1 / sigma^2 * abs(d) / numel(d));
end
```

2.2 CSRT

CSRT is based on channel and spatial reliability. Its general optimisation idea can be expressed as

$$\min_f \sum_{c=1}^C \alpha_c \|S_c(f) * g - y\|^2 + \lambda \|f\|^2,$$

where f is the filter, g is the target template, y is the desired response, $S_c(\cdot)$ represents spatial reliability, and α_c denotes the channel reliability weight^[14]. Its major advantage is better robustness and higher tracking precision, especially in the presence of appearance variation and background interference. Its weakness is the high computational cost caused by richer feature modelling and reliability estimation. In the broader correlation-filter literature, spatial regularisation and complementary reliability modelling have repeatedly been shown to be effective directions for improving robustness^[2,6].

2.3 TLD

TLD contains three modules: tracking, learning, and detection. The tracking module is commonly implemented using Median-Flow to estimate short-term motion and detect tracking failure; the detection module scans the full image by sliding windows and cascaded classifiers; and the learning module updates the detector online using the P-N learning mechanism^[11]. Its advantage is strong long-term recovery ability and better resistance to target disappearance or occlusion. Its disadvantages are high computational cost, strong dependence on successful initialisation, and unstable performance under fast motion or severe disturbance. Related studies that combine recovery-oriented ideas with correlation-filter methods further indicate that hybrid strategies may be valuable when pure short-term tracking is insufficient^[7,18].

2.4 KCF–TLD

In view of the strengths and weaknesses of the above algorithms, this project proposes a KCF–TLD method. Since KCF is much faster than the other two algorithms, KCF is selected as the main tracker. To address the problem that KCF easily loses the target, TLD is used as an auxiliary recovery module. At the beginning of tracking, the target is initialised once; after the target is considered lost, TLD is activated to perform relocalization, and the recovered target box is then used to reinitialise KCF. This design also relates to the broader idea that lightweight tracking and stronger recovery should be allocated to different functional stages rather than being forced into one homogeneous tracker^[2,13].

3 Methodology

3.1 Proposed KCF–TLD Algorithm

The proposed method is described below.

Initialise KCF with the first frame and initial bounding box. For each subsequent frame, first attempt KCF tracking. If the KCF result is reliable, output it and continue. Otherwise, increase the KCF failure counter. Once the failure counter reaches the activation threshold, enter recovery mode and initialise

TLD. During recovery mode, call TLD only at scheduled intervals. Reject recovery candidates that are too far from the last valid target state. Accept a TLD recovery only after repeated and mutually consistent detections. Then reinitialise KCF using the recovered box and return to normal tracking mode. If recovery persists beyond the maximum allowed duration, terminate recovery mode and reset the failure state.

The algorithm spends most of its runtime on high-speed KCF tracking. TLD is called only when KCF becomes unstable or loses the target, so that the hybrid method can improve the success rate while avoiding the heavy cost of running TLD continuously.

3.2 KCF as the Base Tracker

As the base tracker, KCF uses circulant matrices and kernel ridge regression to achieve efficient computation in the frequency domain. Given a base sample x , cyclic shifts of the sample form the training set, and the regression target y is Gaussian-shaped. In each frame, the location with the maximum response is taken as the new target position. This makes KCF highly suitable for real-time UAV tracking.

3.3 TLD as the Recovery Tracker

As the recovery algorithm, TLD is responsible for relocalization after target loss. A recovery candidate is accepted only when it satisfies overlap constraints with the last valid target box and passes continuous consistency checks. After the recovery box is accepted, KCF is reinitialized with the corrected box and the system returns to normal high-speed tracking.

3.4 Hybrid Recovery Strategy

This method does not force KCF and TLD to contribute equally on every frame. Instead, the two algorithms are assigned different roles according to their strengths. KCF is responsible for primary tracking, while TLD is responsible for relocalization after failure. Therefore, the main objective of this design is not simply to make every frame more accurate, but to increase the probability of finding the target again once it has been lost.

3.4.1 Design Logic

The design is based on the complementarity of KCF and TLD. KCF is extremely fast and performs well when frame-to-frame target change is limited. However, once severe occlusion, disappearance, drift, or strong scale change occurs, KCF is often unable to recover by itself. TLD, although significantly slower, has explicit redetection capability. Therefore, the proposed method aims to retain the high FPS advantage of KCF while using TLD as a corrective mechanism when KCF fails.

3.4.2 Triggering TLD

The most important design issue is how to determine that KCF is no longer reliable. In the current implementation, TLD is triggered when the number of consecutive KCF failures or abnormal outputs reaches 2. An output is considered unreliable under two conditions. First, the IoU between the current box and the previous valid box is below 0.15. Second, the area ratio between the current box and the previous valid box falls outside the interval from 0.5 to 2.0. These thresholds are intended to detect obvious spatial discontinuity and sudden scale instability.

Listing 1: Implementation excerpt: KCF unreliability test and TLD activation

```
def _is_kcf_prediction_reliable(self, candidate_bbox):
    if self.last_bbox is None:
        return True

    continuity_iou = bbox_iou(candidate_bbox, self.last_bbox)
    area_ratio = bbox_area(candidate_bbox) / max(bbox_area(self.last_bbox), 1.0)
    if continuity_iou >= self.unstable_continuity_iou_floor:
        return True
    if self.unstable_area_ratio_floor <= area_ratio <= self.unstable_area_ratio_ceiling:
        return True
    return False

if self.kcf_failure_streak < self.tld_activation_failures:
    return False, self.last_bbox if self.last_bbox is not None else (0, 0, 0, 0)
```

```

if self.tld_tracker is None:
    if not self._arm_tld(frame):
        self.last_source = "TLD_INIT_FAIL"
        return False, self.last_bbox if self.last_bbox is not None else (0, 0, 0, 0)
    self.last_source = "TLD_ARMED"
    self.recovery_started_frame = self.frame_counter
    self.last_recovery_attempt_frame = self.frame_counter
    return False, self.last_bbox if self.last_bbox is not None else (0, 0, 0, 0)

```

3.4.3 Controlling TLD Overhead

Since TLD can significantly reduce overall FPS, the algorithm limits how long and how often TLD is allowed to run. In recovery mode, TLD is not executed on every frame, but only once every 3 frames. In addition, the recovery state is protected by a timeout mechanism: the maximum duration of a single recovery phase is 120 frames. Once this limit is exceeded, the recovery state is forcibly terminated so that the system cannot remain trapped in slow recovery indefinitely.

Listing 2: Implementation excerpt: recovery timeout and interval control

```

if self.recovery_started_frame is not None:
    recovery_duration = self.frame_counter - self.recovery_started_frame
    if recovery_duration >= self.recovery_max_frames:
        self.last_source = "RECOVERY_TIMEOUT"
        self.kcf_failure_streak = 0
        self._clear_tld_state()
        return False, self.last_bbox if self.last_bbox is not None else (0, 0, 0, 0)

if self.frame_counter - self.last_recovery_attempt_frame < self.recovery_update_interval_frame:
    self.last_source = "WAIT_RECOVER"
    return False, self.last_bbox if self.last_bbox is not None else (0, 0, 0, 0)

```

3.4.4 Validating TLD Recovery

The algorithm applies three constraints before accepting a TLD recovery box. First, the candidate must have an IoU of at least 0.3 with the last valid target box. Second, the

recovery is not accepted immediately after one detection; instead, it must be confirmed over 2 consecutive detections. Third, the two consecutive candidates must also be sufficiently consistent with each other, with IoU not below 0.45. These checks are designed to reduce false recovery and improve the reliability of relocalization.

Listing 3: Implementation excerpt: recovery candidate confirmation logic

```
recovered_bbox = self._probe_tld(frame, reference_bbox=self.last_bbox)

if recovered_bbox is None:
    self._clear_pending_recovery()
    self.last_source = "LOST"
    return False, self.last_bbox if self.last_bbox is not None else (0, 0, 0, 0)

if self.pending_recovery_bbox is None:
    self.pending_recovery_bbox = recovered_bbox
    self.pending_recovery_hits = 1
    self.last_source = "TLD_CANDIDATE"
    return False, self.last_bbox if self.last_bbox is not None else (0, 0, 0, 0)

consistency_iou = bbox_iou(recovered_bbox, self.pending_recovery_bbox)
if consistency_iou < self.recovery_consistency_iou_floor:
    self.pending_recovery_bbox = recovered_bbox
    self.pending_recovery_hits = 1
    self.last_source = "TLD_RETRY"
    return False, self.last_bbox if self.last_bbox is not None else (0, 0, 0, 0)

self.pending_recovery_bbox = recovered_bbox
self.pending_recovery_hits += 1
if self.pending_recovery_hits < self.recovery_confirmation_frames:
    self.last_source = "TLD_CONFIRM"
    return False, self.last_bbox if self.last_bbox is not None else (0, 0, 0, 0)
```

Listing 4: Implementation excerpt: TLD state reset and arming

```
def _clear_tld_state(self):
    self.tld_tracker = None
```

```

        self._clear_pending_recovery()
        self.recovery_started_frame = None

def _arm_tld(self, frame):
    if self.last_bbox is None:
        return False

    self.tld_tracker = self._init_tracker(self.tld_factory, frame, self.last_bbox)
    self._clear_pending_recovery()
    return self.tld_tracker is not None

```

4 Results and Analysis

4.1 Performance Metrics

This paper evaluates the algorithms mainly through IoU, success, precision, and FPS. The benchmark outputs include a success curve, a precision curve, and an aggregate comparison table. In the present experiment, the comparison particularly focuses on KCF and the proposed KCF-TLD method under the same benchmark setting.

4.2 Algorithm Comparison

The experimental part of the paper should objectively compare KCF, CSRT, TLD, and the proposed KCF-TLD method, with particular emphasis on the proposed method. The purpose is to show, under a unified benchmark, how different tracking principles lead to clear differences in speed, resource demand, robustness, and recovery capability. In the current subset benchmark, KCF-TLD achieves slightly better average IoU, success, and precision than KCF, while also maintaining a higher average FPS in this particular run.

5 Discussion

The discussion focuses on why each algorithm behaves differently. KCF is attractive because of its speed, but it is weak after target loss. CSRT is usually more stable, but slower and more computationally expensive. TLD has explicit recovery capability, but its heavy computation reduces real-time performance. The KCF-TLD design therefore

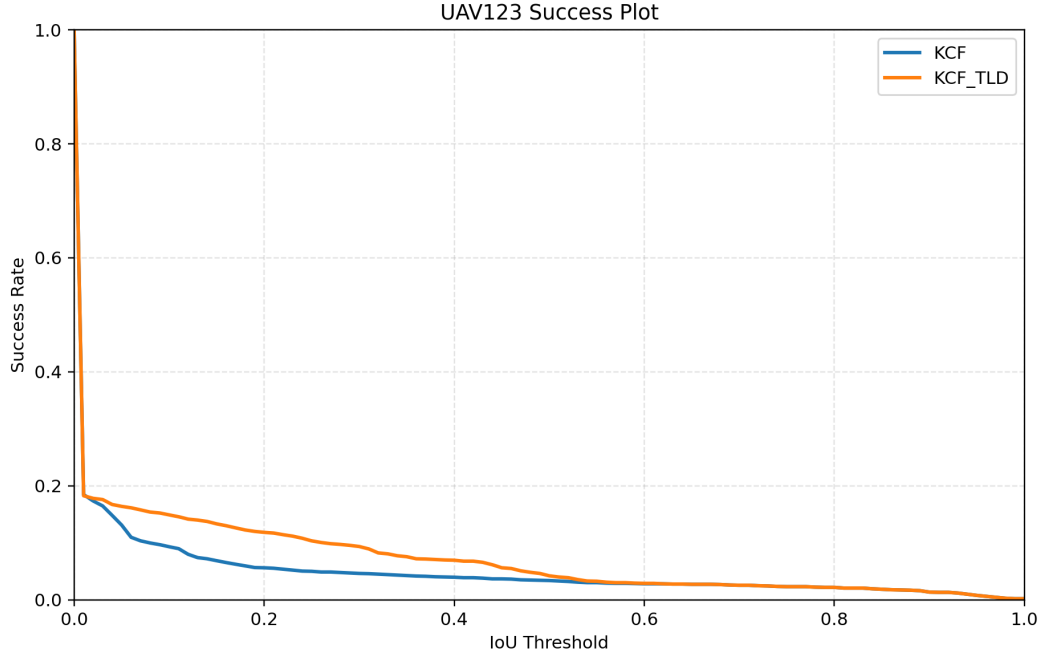


Figure 1: Success plot of KCF and KCF–TLD on the benchmark subset.

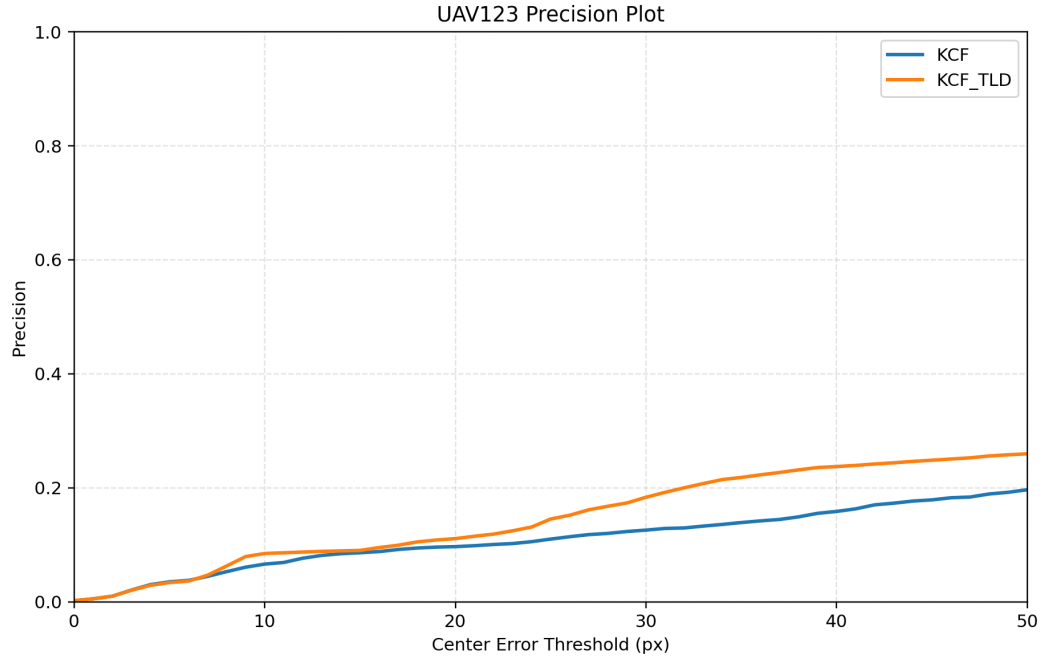


Figure 2: Precision plot of KCF and KCF–TLD on the benchmark subset.

Algorithm	Avg IoU	Success@0.5 (%)	Precision@20px (%)	Avg Center Error	Avg FPS	Seqs
KCF_TLD	0.0651	4.23	11.10	105.209	139.26	3
KCF	0.0445	3.40	9.69	104.440	107.20	3

Figure 3: Aggregate comparison table generated from the benchmark run.

attempts to combine the speed of KCF with the relocalization ability of TLD in a role-based way rather than a full fusion way.

6 Conclusion

This paper proposes a possible way to combine KCF and TLD for UAV visual tracking. The main objective is to solve the problem that KCF cannot recover after target loss. In the proposed method, KCF is used during the normal stage to maintain a high frame rate, while TLD is triggered when KCF is considered unstable or has lost the target. Through continuous confirmation and threshold constraints, the method attempts to improve the reliability of recovery.

Experimental comparison is intended to show that KCF–TLD can improve KCF in terms of IoU, success, precision, and robustness while trying to preserve the high frame rate characteristic of KCF. However, the method still has clear limitations. Since the normal stage is still dominated by KCF, the upper performance bound remains constrained by KCF itself. In addition, TLD recovery is not always stable, so it cannot be treated as a completely reliable module. Future work may improve the instability judgement, strengthen recovery validation, and introduce stronger intelligent models to verify whether the recovered box is consistent with motion direction and short-term scale variation.

References

- [1] BAI H. Shendu Dan Mubiao Zongsuanfa Zongshu [Review of Deep Single-Object Tracking Algorithms][J]. Xiandai Jisuanji, 2021(9): 113-116.
- [2] BERTINETTO L, VALMADRE J, GOLODETZ S, MIKSIK O, TORR P H S. Staple: Complementary Learners for Real-Time Tracking[C/OL]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). 2016: 1401-1409.
- [3] BIBI A, GHANEM B. Multi-Template Scale-Adaptive Kernelized Correlation Filters[C/OL]//Proceedings of the IEEE International Conference on Computer Vision (ICCV) Workshops. 2015: 613-620.

- [4] CHEN X, YANG Z, YU W. An Improved KCF Mixed with Deep Feature for Pedestrian Tracking[J/OL]. Journal of Physics: Conference Series, 2023, 2465(1): 012015.
- [5] DANELLJAN M, BHAT G, KHAN F S, FELSBURG M. ATOM: Accurate Tracking by Overlap Maximization[C/OL]//Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). 2019: 4660-4669.
- [6] DANELLJAN M, HAGER G, KHAN F S, FELSBURG M. Learning Spatially Regularized Correlation Filters for Visual Tracking[C/OL]//Proceedings of the IEEE International Conference on Computer Vision (ICCV). 2015: 4310-4318.
- [7] FAN J, SHEN X, WU Y. Long-Term Visual Tracking Algorithm for UAVs Based on Kernel Correlation Filtering and SURF Features[J/OL]. The Visual Computer, 2022, 38(3): 1043-1057.
- [8] GAO J, ZHANG T, YANG X, XU C. Deep Relative Tracking[J/OL]. IEEE Transactions on Image Processing, 2019, 28(12): 5987-6002.
- [9] GUO Q, FENG W, ZHOU C, HUANG R, WAN L, WANG S. Learning Dynamic Siamese Network for Visual Object Tracking[C/OL]//Proceedings of the IEEE International Conference on Computer Vision (ICCV). 2017: 1781-1789.
- [10] HENRIQUES J F, CASEIRO R, MARTINS P, BATISTA J. High-Speed Tracking with Kernelized Correlation Filters[J/OL]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2015, 37(3): 583-596.
- [11] KALAL Z, MIKOLAJCZYK K, MATAS J. Tracking-Learning-Detection[J/OL]. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2012, 34(7): 1409-1422.
- [12] KRISTAN M, MATAS J, LEONARDIS A, et al. The Seventh Visual Object Tracking VOT2019 Challenge Results[C/OL]//Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV) Workshops. 2019: 2206-2241.
- [13] LI Y, ZHU J, HOI S C H. Reliable Patch Trackers: Robust Visual Tracking by Exploiting Reliable Patches[C/OL]//Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR). 2015: 353-361.
- [14] LUKEŽIĆ A, VOJÍŘ T, ZAJC L Č, MATAS J, KRISTAN M. Discriminative Correlation Filter with Channel and Spatial Reliability[C/OL]//2017 IEEE Conference

- on Computer Vision and Pattern Recognition (CVPR). 2017: 4847-4856.
- [15] MBOUOMBOUO MBOUNGAM A H, WANG X, LI Y. Real-Time Tracking Target System Based on Kernelized Correlation Filter in Complicated Areas[EB/OL]. 2024 [2026-04-09]. <https://core.ac.uk/download/pdf/XXXXXX.pdf>.
 - [16] TANG D, JIN W, LIU D, CHE J, YANG Y. Siam Deep Feature KCF Method and Experimental Study for Pedestrian Tracking[J/OL]. Electronics, 2023, 12(3): 669.
 - [17] WANG C, ZHANG L, XIE L, YUAN J. Kernelized Correlation Filter Tracking with Scale Adaptation and Spatial Reliability[J/OL]. IEEE Access, 2018, 6: 50748-50758.
 - [18] ZHANG J, HUANG H, WANG J, BAO J. Ronghe CN Genzong Suanfa Gaijin de TLD Shishi Mubiao Genzong Suanfa [Improved Real-Time TLD Tracking Algorithm Based on CN Tracking Fusion][J]. Jisuanji Gongcheng yu Kexue, 2020, 42(7): 1215-1225.
 - [19] ZHANG K, ZHANG L, LIU Q, ZHANG D, YANG M H. Fast Visual Tracking via Dense Spatio-Temporal Context Learning[C/OL]//Proceedings of the European Conference on Computer Vision (ECCV). 2014: 127-141.
 - [20] ZHANG L, SUGANTHAN P N. Visual Tracking with Convolutional Neural Network and Kernelized Correlation Filters[J/OL]. Journal of Visual Communication and Image Representation, 2017, 45: 174-183.